



ST JOSEPH'S SCHOOL

PROJECT REPORT ON

GAME DEVELOPMENT

**A project submitted in partial fulfillment of Computer
Science Practical Examination**

**Class XII
2024-25**

SUBJECT : Computer Science

SUBJECT CODE : 083

Student Name : Katherine Kollannoor

Class & Section : 12 - A

Teacher's Name : Ms. Yashaswini Sujay

ACKNOWLEDGMENT

I would like to express my gratitude to
Ms Yashaswini Sujay for her able
guidance and support in completing my project.

I would also like to extend my gratitude to my Principal & Coordinator for
providing me with all the required facilities.

Lastly, I would also like to thank my parents and
friends without whose moral support I would not have
been able to complete this project.

Katherine Kollannoor

TABLE OF CONTENTS

SR. NO.	TOPIC	PAGE NO.
1.	Preface	1
2.	Introduction	2
3.	Objective of the Project	3
4.	Scope of the Project	4
5.	Hardware and Software Requirement	5
6.	System Design and Development	7
7.	Conclusion	25
8.	Bibliography	26

PREFACE

Our project is based on the card game "Concentration" (also known as "Memory"). In the game, the first player is chosen by any method. The player then selects two cards to flip over, leaving the first card face up until the second card is revealed. If the two cards form a matching pair, the player takes them and places them face down in a pile, then turns over two more cards. If the cards do not form a pair, the player must wait for at least five seconds before turning them face down again, keeping them in their original positions on the table. The turn then passes to the next player (usually the player to the left). The game continues in this manner until all the cards are matched.

The idea for our project came about one day during our games period in class, when we were playing "Concentration" and having a great time. When we were later assigned to create a GUI project, we immediately thought of this game. We realized it would be both a fun and challenging project, so we decided to pursue it and quickly shared our idea with our teacher, who supported the plan.

INTRODUCTION

After playing memory games like "Concentration," students often experience improved recall of specific names, terms, and functions. These games require concentration, pattern recognition, and attention to detail, all of which stimulate cognitive processes. Engaging in such activities not only enhances memory retention but also boosts overall brain function and activity. Studies have shown that memory games help strengthen neural connections, particularly in areas related to short-term and long-term memory. As a result, students become better at recalling information quickly and accurately, which can have a positive impact on their learning abilities across various subjects.

In addition to improving memory, these games also foster critical thinking skills, as players must analyze patterns and make strategic decisions. The mental challenge involved in matching cards strengthens cognitive flexibility and problem-solving abilities, which are essential skills both in and outside of the classroom. By engaging with games like "Concentration," students develop a fun yet effective way to enhance their cognitive abilities, all while honing skills that will benefit them in academic and real-world situations.

OBJECTIVE OF THE PROJECT

1. Goals to be Achieved by This Project

- Create a Functional GUI: Design and implement a user-friendly graphical interface for the game, making it accessible to players of various age groups.
- Improve Cognitive Skills: Encourage players to improve their memory, attention, and concentration through repeated gameplay, as they must remember card positions and recognize patterns.
- Provide Educational Value: Offer an enjoyable and challenging game that can be used as a tool for educational purposes, promoting better retention and recall in students.
- Track Performance: Implement features to track player progress, including a live score counter, turn indicators for each player, and a leaderboard that displays the number of wins for each player.

2. To Whom the Project Will Be Helpful & How It Will Be Helpful

- Students: The game helps students enhance memory, concentration, and cognitive skills in a fun, engaging way.
- Teachers and Educators: Educators can use the game to reinforce lessons and strengthen students' memorization and problem-solving skills. It provides an interactive method for improving focus and academic performance.
- Parents: Parents can support their children's cognitive development by encouraging gameplay at home. The game offers a non-stressful, enjoyable way for children to practice memory and problem-solving skills.
- Seniors: The game helps seniors maintain mental agility by engaging them in memory exercises. Regular play can support memory retention and cognitive function in a fun, low-pressure environment.

3. How the Records Will Be Managed

- Player Input: Players will be prompted to enter their names, which will then be securely stored in a MySQL table called 'memory' within the 'card_game' database.
- Winner Tracking: After each game, the name of the winner will be recorded in the same 'memory' table in the 'card_game' database.
- Leaderboard Generation: Utilizing the data stored in the 'memory' table, we will generate a leaderboard that ranks players by the number of wins

SCOPE OF THE PROJECT

Currently, the game uses 8 pairs of cards and supports 2 players. To enhance the user experience and increase replayability, we can introduce customization features that allow players to:

- **Choose the Number of Card Pairs:** Players can select the number of pairs they wish to play with, ranging from a smaller, faster-paced game to a more challenging, larger set of pairs.
- **Adjust the Number of Players:** Support for more than two players can be added, allowing for multiplayer gameplay with customizable team sizes or competitive play.
- **Optional Game Modes:** We could introduce different game modes such as "timed rounds," "memory challenges," or "team play," offering a variety of playstyles to suit different preferences.
- **Replay Button:** Currently, the game ends directly after a match concludes. Adding a "Replay" button would give players the option to quickly restart the game without having to navigate back to the main menu. This feature would streamline the flow of gameplay and encourage more frequent playthroughs.

HARDWARE AND SOFTWARE REQUIREMENT

1. Minimum Requirements:

These are the basic requirements to run the game, though performance may be limited.

- Hardware:
 - Processor: Intel Core i3 or equivalent
 - RAM: 4 GB
 - Storage: 100 MB free disk space
 - Display: 1024 x 768 resolution
 - Graphics: Integrated graphics
 - Input Device: Mouse and keyboard
- Software:
 - OS: Windows 7+, macOS 10.12+, or Linux
 - Python: 3.6 or higher
 - Libraries: Tkinter, MySQL Connector, Pillow
 - Database: MySQL 5.7+ (local or remote)

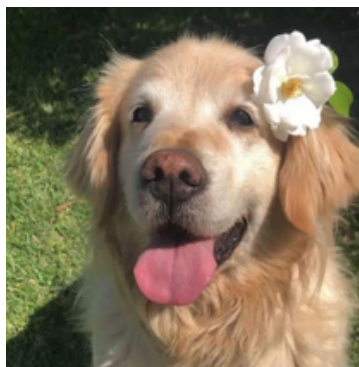
2. Recommended Requirements:

These specs provide a smoother experience with better performance.

- Hardware:
 - Processor: Intel Core i5 or equivalent
 - RAM: 8 GB
 - Storage: 500 MB free disk space
 - Display: 1920 x 1080 resolution
 - Graphics: Dedicated GPU (optional)
 - Input Device: Mouse and keyboard
- Software:
 - OS: Windows 10+, macOS 10.14+, or modern Linux
 - Python: 3.8 or higher
 - Libraries: Tkinter, MySQL Connector, Pillow
 - Database: MySQL 8.0+ (local or cloud)

3. Images required:

This game requires 9 images, titled '1.png', '2.png', ... , '8.png' and 'back.png'.



SYSTEM DESIGN AND DEVELOPMENT

FLOWCHART

CONCENTRATION

OPENING WINDOW

Labels:

1. A label displaying the name of the game "CONCENTRATION" prominently at the top of the window.

Buttons:

1. A button labeled "START" that, when clicked, navigates to the next window.

PLAYER INPUT WINDOW

Labels:

1. A label displaying the text "ENTER PLAYER NAMES" at the top of the window.
2. A label with the text "Player 1:" to prompt the first player's name.
3. A label with the text "Player 2:" to prompt the second player's name.

Entry Widgets:

1. placed next to Player 1, where player 1 can type in their name.
2. placed next to Player 2, where player 2 can type in their name.

Buttons:

1. A button labeled "PLAY" that, when clicked, navigates to the next window to start the game.

MEMORY GAME WINDOW

Labels:

1. "PLAYERS", a header to identify the section where the player names are displayed.
2. Player 1's Name, displays the name of Player 1 as entered by the user in the previous window.
3. Player 2's Name, displays the name of Player 2 as entered by the user in the previous window.
4. "SCORES", a header to identify the section showing the current scores.
5. Player 1's Score, a label showing Player 1's current score, which updates in real-time as the game progresses.
6. Player 2's Score, a label showing Player 2's current score, which also updates dynamically.
7. "TURN: [Player Name]", displays the name of the player whose turn it is. This label updates after each move.
8. "WINNER IS: [Winner Name]", this label appears only once the game is over, showing the name of the winner.

Buttons:

1. Buttons 1-16, these buttons represent the memory cards the players will click to reveal and match pairs. Initially, each button is blank, indicating the card is face-down. When clicked, the card reveals its image.
2. "NEXT" Button, when clicked, this button advances the game to the next player's turn.
3. "FINISH" Button, once the game is over, the "NEXT" button is replaced with a "FINISH" button, which allows the player to finish the game and proceed to the next screen.

POST GAME WINDOW

Buttons:

1. "LEADERBOARD", a button that, when clicked, takes the player to a new window.
2. "END", a button that closes the game and exits the application when clicked.

LEADERBOARD WINDOW

Labels:

1. "LEADERBOARD", the title for the overall leaderboard section.
2. "WINNER NAME", a label under which the named of each winner is displayed.
3. "NUMBER OF WINS", a label under which the number of wins corresponding to each winner is displayed.

Buttons:

1. "DONE", a button that closes the game and exits the application when clicked.

SYSTEM DESIGN AND DEVELOPMENT

DATABASE DESIGN

Database Name: card_game

Table Name: memory

Fields:

1. sr_no: Stores the unique game number.
2. player1: Stores the name of the first player.
3. player2: Stores the name of the second player.
4. winner: Stores the name of the player who won the game.

```
mysql> use card_game;
Database changed
mysql> desc memory;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sr_no | int           | YES  |     | NULL    |       |
| player1 | varchar(20)   | YES  |     | NULL    |       |
| player2 | varchar(20)   | YES  |     | NULL    |       |
| winner  | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> select * from memory;
+-----+-----+-----+-----+
| sr_no | player1 | player2 | winner |
+-----+-----+-----+-----+
| 1     | Dhakshika | Sandrina | Dhakshika |
| 2     | Katherine | Dhakshika | tie |
| 3     | Katherine | Dhakshika | Katherine |
| 4     | Sandrina | Dhakshika | Dhakshika |
| 5     | Katherine | Sandrina | Sandrina |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

SYSTEM DESIGN AND DEVELOPMENT

CODING

memory game - Copy.py - C:\Users\DELL38\Desktop\KATHERINE XII-A 24-25\memory ga... — □ ×

File Edit Format Run Options Window Help

```
1 import tkinter as tk
2 from tkinter import *
3 from PIL import Image,ImageTk
4 import random as r
5 import mysql.connector as ms
6 con=ms.connect(host="localhost",user="root",password="root123")
7 cur=con.cursor()
8
9 flag=0; selected1=""; selected2=""; turn=0; scoreP1=0; scoreP2=0; winner = ''
10 pl1=""; pl2=""; set1Error=""; set2Error=""
11
12 #making window 1
13 window1=Tk()
14 window1.title('opening screen')
15 window1.geometry('800x500')
16 window1.configure(bg = 'light yellow')
17 wel_msg=tk.Label(window1, text = 'CONCENTRATION !!', font = ('Calibri', 60, 'bold'),
18                 fg = '#f7be00', bg = 'light yellow').pack(pady = 75)
19
20 def open_window2():
21     #making window 2
22     window1.destroy()
23     window2 = Tk()
24     window2.title("player input")
25     window2.geometry('750x500')
26     window2.configure(bg = 'light blue')
27
28     #to only allow alphabets
29     def on_validate_input(P):
30         if P == "" or P.isalpha():
31             return True
32         return False
33     vcmd = window2.register(on_validate_input)
34
35     playerLabel = Label(window2, text = 'ENTER PLAYER NAMES', font = ('Calibri', 50, 'bold'),
36                         fg = '#00537f', bg = 'light blue').place(x = 50, y = 50)
37     plentryLabel = Label(window2, text = 'player 1:', font = ('Calibri', 50), fg = 'black',
38                         bg = 'light blue').place(x = 50, y = 150)
39     plentry = tk.StringVar()
40     p1=Entry(window2, font=('Calibri',20), textvariable = plentry, validate='key',
41             validatecommand=(vcmd, '%P')).place(x = 350, y = 170, height = 50, width = 300)
42     p2entryLabel = Label(window2, text = 'player 2:', font = ('Calibri', 50), fg = 'black',
43                         bg = 'light blue').place(x = 50, y = 250)
44     p2entry = tk.StringVar()
45     p2=Entry(window2, font=('Calibri',20), textvariable = p2entry, validate='key',
46             validatecommand=(vcmd, '%P')).place(x = 350, y = 270, height = 50, width = 300)
```

```

47
48 #not null constraint
49 def constraint():
50     if(p1entry.get()==" or p2entry.get()==""):
51         constraint=Tk()
52         constraint.title('Error')
53         constraint.geometry('365x100')
54         errorLabel=Label(constraint, text='Warning: names cannot be empty',
55                             font = ('Arial', 17), foreground='red').place(x = 10, y = 10)
56         def tryAgainButtonClick():
57             constraint.destroy()
58             tryAgainButton = Button(constraint, text="Try Again", command=tryAgainButtonClick,
59                                     foreground='black', background='light grey').place(x = 150, y = 55)
60             constraint.mainloop()
61     else:
62         open_window3()
63
64 def open_window3():
65     global c
66     #adding sr no and players to mysql table
67     p1=p1entry.get()
68     p2=p2entry.get()
69     cur.execute("show databases")
70     data=cur.fetchall()
71     if(("card_game",) not in data):
72         cur.execute("create database card_game")
73     cur.execute("use card_game")
74     cur.execute("show tables")
75     data=cur.fetchall()
76     if(("memory",) not in data):
77         cur.execute("create table memory(sr_no int, player1 varchar(20) , player2 varchar(20),"
78                                     "winner varchar(20))")
79     cur.execute("select sr_no from memory")
80     data=cur.fetchall()
81     c=len(data)+1
82     cur.execute("insert into memory(sr_no,player1,player2) values({}, '{}', '{}')".format(c,p1,p2))
83     con.commit()
84
85     #making window 3
86     window2.destroy()
87     window3=Tk()
88     window3.title("memory game")
89     window3.geometry('900x500')
90     window3.configure(bg = 'white')
91
92     #stuff in window 3 [leftside , cards]
93     card_back=PhotoImage(file='back.png')
94     l=[1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8]
95

```

```

96     b1 = r.choice(l)
97     l.remove(b1)
98     c1=PhotoImage(file=b1+'.png')
99     def click1():
100         global flag, selected1, selected2
101         flag+=1
102         if flag==1:
103             selected1=b1
104         if flag==2:
105             selected2=b1
106         if flag<=2:
107             button1=Button(window3, image=c1).place(x = 10, y = 10, height =100, width = 100)
108     button1=Button(window3, image=card_back, command = click1).place(x = 10, y = 10,
109                             height =100, width = 100)
110
111     b2 = r.choice(l)
112     l.remove(b2)
113     c2=PhotoImage(file=b2+'.png')
114     def click2():
115         global flag, selected1, selected2
116         flag+=1
117         if flag==1:
118             selected1=b2
119         if flag==2:
120             selected2=b2
121         if flag<=2:
122             button2=Button(window3, image=c2).place(x = 135, y = 10, height =100, width = 100)
123     button2=Button(window3, image=card_back, command = click2).place(x = 135, y = 10,
124                             height =100, width = 100)
125
126     b3 = r.choice(l)
127     l.remove(b3)
128     c3=PhotoImage(file=b3+'.png')
129     def click3():
130         global flag, selected1, selected2
131         flag+=1
132         if flag==1:
133             selected1=b3
134         if flag==2:
135             selected2=b3
136         if flag<=2:
137             button3=Button(window3, image=c3).place(x = 260, y = 10, height =100, width = 100)
138     button3=Button(window3, image=card_back, command = click3).place(x = 260, y = 10,
139                             height =100, width = 100)
140

```

```

141     b4 = r.choice(l)
142     l.remove(b4)
143     c4=PhotoImage(file=b4+'.png')
144     def click4():
145         global flag, selected1, selected2
146         flag+=1
147         if flag==1:
148             selected1=b4
149         if flag==2:
150             selected2=b4
151         if flag<=2:
152             button4=Button(window3, image=c4).place(x = 385, y = 10, height =100, width = 100)
153     button4=Button(window3, image=card_back, command = click4).place(x = 385, y = 10,
154                             height =100, width = 100)
155
156     b5 = r.choice(l)
157     l.remove(b5)
158     c5=PhotoImage(file=b5+'.png')
159     def click5():
160         global flag, selected1, selected2
161         flag+=1
162         if flag==1:
163             selected1=b5
164         if flag==2:
165             selected2=b5
166         if flag<=2:
167             button5=Button(window3, image=c5).place(x = 10, y = 135, height =100, width = 100)
168     button5=Button(window3, image=card_back, command = click5).place(x = 10, y = 135,
169                             height =100, width = 100)
170
171     b6 = r.choice(l)
172     l.remove(b6)
173     c6=PhotoImage(file=b6+'.png')
174     def click6():
175         global flag, selected1, selected2
176         flag+=1
177         if flag==1:
178             selected1=b6
179         if flag==2:
180             selected2=b6
181         if flag<=2:
182             button6=Button(window3, image=c6).place(x = 135, y = 135, height =100, width = 100)
183     button6=Button(window3, image=card_back, command = click6).place(x = 135, y = 135,
184                             height =100, width = 100)
185

```

```

186     b7 = r.choice(l)
187     l.remove(b7)
188     c7=PhotoImage(file=b7+'.png')
189     def click7():
190         global flag, selected1, selected2
191         flag+=1
192         if flag==1:
193             selected1=b7
194         if flag==2:
195             selected2=b7
196         if flag<=2:
197             button7=Button(window3, image=c7).place(x = 260, y = 135, height =100, width = 100)
198     button7=Button(window3, image=card_back, command = click7).place(x = 260, y = 135,
199                             height =100, width = 100)
200
201     b8 = r.choice(l)
202     l.remove(b8)
203     c8=PhotoImage(file=b8+'.png')
204     def click8():
205         global flag, selected1, selected2
206         flag+=1
207         if flag==1:
208             selected1=b8
209         if flag==2:
210             selected2=b8
211         if flag<=2:
212             button8=Button(window3, image=c8).place(x = 385, y = 135, height =100, width = 100)
213     button8=Button(window3, image=card_back, command = click8).place(x = 385, y = 135,
214                             height =100, width = 100)
215
216     b9 = r.choice(l)
217     l.remove(b9)
218     c9=PhotoImage(file=b9+'.png')
219     def click9():
220         global flag, selected1, selected2
221         flag+=1
222         if flag==1:
223             selected1=b9
224         if flag==2:
225             selected2=b9
226         if flag<=2:
227             button9=Button(window3, image=c9).place(x = 10, y = 260, height =100, width = 100)
228     button9=Button(window3, image=card_back, command = click9).place(x = 10, y = 260,
229                             height =100, width = 100)
230

```

```

231 b10 = r.choice(l)
232 l.remove(b10)
233 c10=PhotoImage(file=b10+'.png')
234 def click10():
235     global flag, selected1, selected2
236     flag+=1
237     if flag==1:
238         selected1=b10
239     if flag==2:
240         selected2=b10
241     if flag<=2:
242         button10=Button(window3, image=c10).place(x = 135, y = 260, height =100, width = 100)
243 button10=Button(window3, image=card_back, command = click10).place(x = 135, y = 260,
244                             height =100, width = 100)
245
246 b11 = r.choice(l)
247 l.remove(b11)
248 c11=PhotoImage(file=b11+'.png')
249 def click11():
250     global flag, selected1, selected2
251     flag+=1
252     if flag==1:
253         selected1=b11
254     if flag==2:
255         selected2=b11
256     if flag<=2:
257         button11=Button(window3, image=c11).place(x = 260, y = 260, height =100, width = 100)
258 button11=Button(window3, image=card_back, command = click11).place(x = 260, y = 260,
259                             height =100, width = 100)
260
261 b12 = r.choice(l)
262 l.remove(b12)
263 c12=PhotoImage(file=b12+'.png')
264 def click12():
265     global flag, selected1, selected2
266     flag+=1
267     if flag==1:
268         selected1=b12
269     if flag==2:
270         selected2=b12
271     if flag<=2:
272         button12=Button(window3, image=c12).place(x = 385, y = 260, height =100, width = 100)
273 button12=Button(window3, image=card_back, command = click12).place(x = 385, y = 260,
274                             height =100, width = 100)
275

```

```

276     b13 = r.choice(l)
277     l.remove(b13)
278     c13=PhotoImage(file=b13+'.png')
279     def click13():
280         global flag, selected1, selected2
281         flag+=1
282         if flag==1:
283             selected1=b13
284         if flag==2:
285             selected2=b13
286         if flag<=2:
287             button13=Button(window3, image=c13).place(x = 10, y = 385, height =100, width = 100)
288     button13=Button(window3, image=card_back, command = click13).place(x = 10, y = 385,
289                                     height =100, width = 100)
290
291     b14 = r.choice(l)
292     l.remove(b14)
293     c14=PhotoImage(file=b14+'.png')
294     def click14():
295         global flag, selected1, selected2
296         flag+=1
297         if flag==1:
298             selected1=b14
299         if flag==2:
300             selected2=b14
301         if flag<=2:
302             button14=Button(window3, image=c14).place(x = 135, y = 385, height =100, width = 100)
303     button14=Button(window3, image=card_back, command = click14).place(x = 135, y = 385,
304                                     height =100, width = 100)
305
306     b15 = r.choice(l)
307     l.remove(b15)
308     c15=PhotoImage(file=b15+'.png')
309     def click15():
310         global flag, selected1, selected2
311         flag+=1
312         if flag==1:
313             selected1=b15
314         if flag==2:
315             selected2=b15
316         if flag<=2:
317             button15=Button(window3, image=c15).place(x = 260, y = 385, height =100, width = 100)
318     button15=Button(window3, image=card_back, command = click15).place(x = 260, y = 385,
319                                     height =100, width = 100)
320

```

```

321 b16 = r.choice(l)
322 l.remove(b16)
323 c16=PhotoImage(file=b16+'.png')
324 def click16():
325     global flag, selected1, selected2
326     flag+=1
327     if flag==1:
328         selected1=b16
329     if flag==2:
330         selected2=b16
331     if flag<=2:
332         button16=Button(window3, image=c16).place(x = 385, y = 385, height =100, width = 100)
333 button16=Button(window3, image=card_back, command = click16).place(x = 385, y = 385,
334                             height =100, width = 100)
335
336 #cheatcode for game
337 print(b1,b2,b3,b4)
338 print(b5,b6,b7,b8)
339 print(b9,b10,b11,b12)
340 print(b13,b14,b15,b16)
341
342 #stuff in window 3 [rightside , widgits]
343 global turn, scoreP1, scoreP2
344 players = Label(window3, text = 'PLAYERS', font = ('Calibri', 20, 'bold'), fg = 'black',
345                 bg = 'white').place(x = 550, y = 20)
346 scores = Label(window3, text = 'SCORES', font = ('Calibri', 20, 'bold'), fg = 'black',
347               bg = 'white').place(x = 715, y = 20)
348 turns = Label(window3, text = 'TURN: ', font = ('Calibri', 20, 'bold'), fg = 'black',
349              bg = 'white').place(x = 550, y = 200)
350 P1Label = Label(window3, text = p11, font = ('Calibri', 15, 'bold'), fg = 'red',
351               bg = 'white').place(x = 550, y = 75)
352 P2Label = Label(window3, text = p12, font = ('Calibri', 15, 'bold'), fg = 'green',
353               bg = 'white').place(x = 550, y = 125)
354 S1Label = Label(window3, text = scoreP1, font = ('Calibri', 15), fg = 'black',
355               bg = 'white').place(x = 750, y = 75)
356 S2Label =Label(window3, text = scoreP2, font = ('Calibri', 15), fg = 'black',
357               bg = 'white').place(x = 750, y = 125)
358 turn=1
359 turnLabel = Label(window3, text = p11+"s turn", font = ('Calibri', 20, 'bold'),
360                  fg = 'red', bg = 'white').place(x = 650, y = 200)
361
362 def nextButtonClick():
363     global flag, selected1, selected2, turn, scoreP1, scoreP2, c
364     #if match is made correctly
365     if flag>=2 and selected1!=" and selected1==selected2:
366         if turn%2==0:
367             scoreP2+=1
368             Label(window3, text = scoreP2, font = ('Calibri', 15, ), fg = 'black',
369                   bg = 'white').place(x = 750, y = 125)
370         else:
371             scoreP1+=1
372             Label(window3, text = scoreP1, font = ('Calibri', 15, ), fg = 'black',
373                   bg = 'white').place(x = 750, y = 75)
374     flag=0; selected1=""; selected2=""

```

```

375
376 #if match is made incorrectly
377 if flag>=2 and selected1!=" and selected1!=selected2:
378     turn+=1
379     if turn%2==0:
380         turnLabel = Label(window3, text = pl1+"s turn", font = ('Calibri', 20, 'bold'),
381                             fg = 'white', bg = 'white').place(x = 650, y = 200)
382         turnLabel = Label(window3, text = pl2+"s turn", font = ('Calibri', 20, 'bold'),
383                             fg = 'green', bg = 'white').place(x = 650, y = 200)
384     else:
385         turnLabel = Label(window3, text = pl2+"s turn", font = ('Calibri', 20, 'bold'),
386                             fg = 'white', bg = 'white').place(x = 650, y = 200)
387         turnLabel = Label(window3, text = pl1+"s turn", font = ('Calibri', 20, 'bold'),
388                             fg = 'red', bg = 'white').place(x = 650, y = 200)
389
390 if selected1 == b1 or selected2 == b1:
391     button1=Button(window3, image=card_back, command = click1).place(x = 10,
392                             y = 10, height =100, width = 100)
393 if selected1 == b2 or selected2 == b2:
394     button2=Button(window3, image=card_back, command = click2).place(x = 135,
395                             y = 10, height =100, width = 100)
396 if selected1 == b3 or selected2 == b3:
397     button3=Button(window3, image=card_back, command = click3).place(x = 260,
398                             y = 10, height =100, width = 100)
399 if selected1 == b4 or selected2 == b4:
400     button4=Button(window3, image=card_back, command = click4).place(x = 385,
401                             y = 10, height =100, width = 100)
402 if selected1 == b5 or selected2 == b5:
403     button5=Button(window3, image=card_back, command = click5).place(x = 10,
404                             y = 135, height =100, width = 100)
405 if selected1 == b6 or selected2 == b6:
406     button6=Button(window3, image=card_back, command = click6).place(x = 135,
407                             y = 135, height =100, width = 100)
408 if selected1 == b7 or selected2 == b7:
409     button7=Button(window3, image=card_back, command = click7).place(x = 260,
410                             y = 135, height =100, width = 100)
411 if selected1 == b8 or selected2 == b8:
412     button8=Button(window3, image=card_back, command = click8).place(x = 385,
413                             y = 135, height =100, width = 100)
414 if selected1 == b9 or selected2 == b9:
415     button9=Button(window3, image=card_back, command = click9).place(x = 10,
416                             y = 260, height =100, width = 100)
417 if selected1 == b10 or selected2 == b10:
418     button10=Button(window3, image=card_back, command = click10).place(x = 135,
419                             y = 260, height =100, width = 100)
420 if selected1 == b11 or selected2 == b11:
421     button11=Button(window3, image=card_back, command = click11).place(x = 260,
422                             y = 260, height =100, width = 100)
423 if selected1 == b12 or selected2 == b12:
424     button12=Button(window3, image=card_back, command = click12).place(x = 385,
425                             y = 260, height =100, width = 100)
426 if selected1 == b13 or selected2 == b13:
427     button13=Button(window3, image=card_back, command = click13).place(x = 10,
428                             y = 385, height =100, width = 100)
429 if selected1 == b14 or selected2 == b14:
430     button14=Button(window3, image=card_back, command = click14).place(x = 135,
431                             y = 385, height =100, width = 100)

```

```

432     if selected1 == b15 or selected2 == b15:
433         button15=Button(window3, image=card_back, command = click15).place(x = 260,
434                                     y = 385, height =100, width = 100)
435     if selected1 == b16 or selected2 == b16:
436         button16=Button(window3, image=card_back, command = click16).place(x = 385,
437                                     y = 385, height =100, width = 100)
438     flag=0; selected1=""; selected2=""
439
440     #when all matches are made
441     if scoreP1+scoreP2==8:
442         if scoreP1>scoreP2:
443             winnerLabel = Label(window3, text="WINNER IS "+p1.upper(),
444                                 font=('Calibri', 25 , 'bold'), fg='gold', bg='white').place(x=550, y=320)
445             cur.execute("update memory set winner = '{}' where sr_no={}".format(p1,c))
446         elif scoreP2>scoreP1:
447             winnerLabel = Label(window3, text="WINNER IS "+p2.upper(),
448                                 font=('Calibri', 25 , 'bold'), fg='gold', bg='white').place(x=550, y=320)
449             cur.execute("update memory set winner = '{}' where sr_no={}".format(p2,c))
450         else:
451             winnerLabel = Label(window3, text="THIS GAME IS A DRAW",
452                                 font=('Calibri', 25 , 'bold'), fg='black', bg='white').place(x=500, y=320)
453             cur.execute("update memory set winner = '{}' where sr_no={}".format('tie',c))
454     con.commit()
455
456     def open_window4():
457         #making window 4
458         window3.destroy()
459         window4=Tk()
460         window4.title('post game screen')
461         window4.geometry('400x200')
462         window4.configure(bg = '#ffc1c1')
463
464         def leaderboard():
465             window4.destroy()
466             lb=Tk()
467             lb.title('leaderboard')
468             lb.configure(bg = 'light green')
469             cur.execute("select winner from memory")
470             winner_all=cur.fetchall()
471             cur.execute("select distinct winner from memory where winner is not null")
472             winner_distinct=cur.fetchall()
473             ht=len(winner_distinct)
474             lb.geometry('580x'+str(240+(ht*50)))
475
476             l=[]
477             for i in winner_distinct:
478                 l.append([i,winner_all.count(i)])
479
480             for i in range(0,len(l) , 1):
481                 for j in range(0,len(l) , 1):
482                     if(l[j][1]<l[j+1][1]):
483                         l[j],l[j+1]=l[j+1],l[j]

```

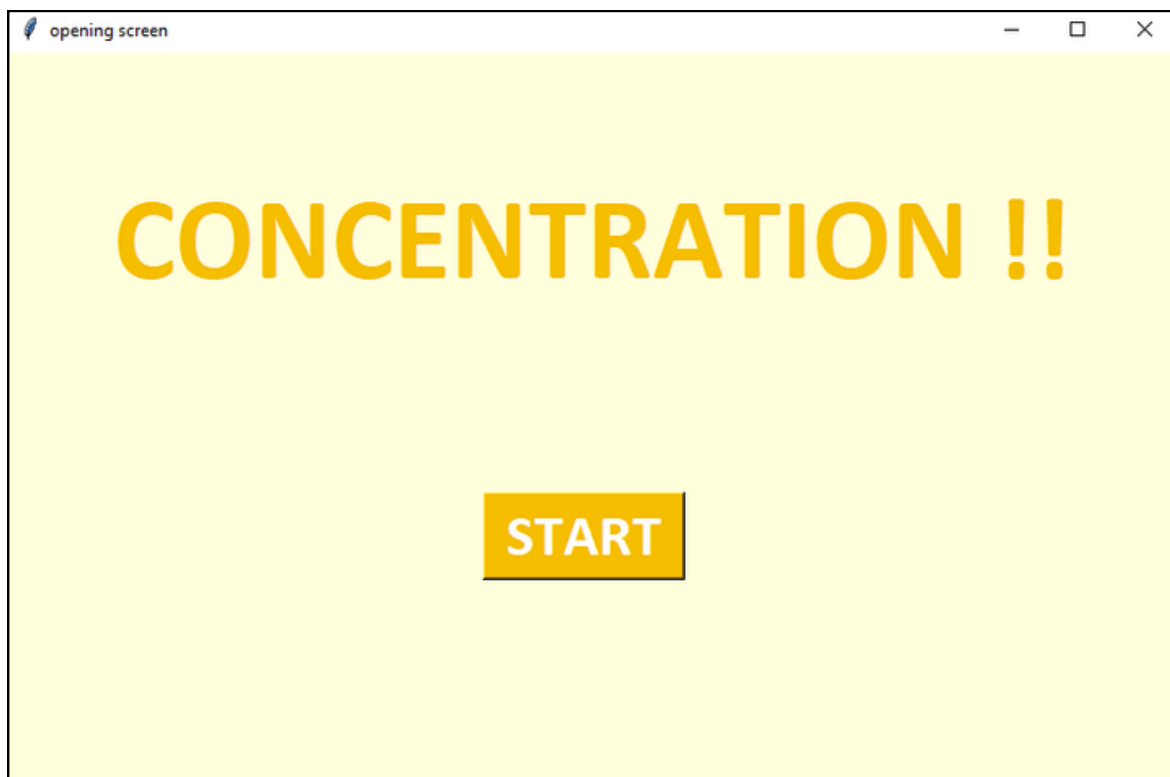
```

484 lbLabel = Label(lb, text="LEADERBOARD", font = ('Calibri', 50, 'bold'),
485               fg = '#1d5e2f', bg = 'light green').place(x = 70, y = 10)
486 winnerLabel = Label(lb, text="WINNER NAME", font = ('Calibri', 20, 'bold'),
487                  fg = 'white', bg = 'light green').place(x = 60, y = 100)
488 winsLabel = Label(lb, text="NUMBER OF WINS", font = ('Calibri', 20, 'bold'),
489                 fg = 'white', bg = 'light green').place(x = 300, y = 100)
490 y1 = 150
491 for i in range(0, len(l)):
492     if l[i][0][0] == 'tie':
493         continue
494     winnerNameLabel = Label(lb, text=l[i][0][0], font = ('Calibri', 15), fg = 'black',
495                      bg = 'light green').place(x = 80, y = y1)
496     winsCountLabel = Label(lb, text=l[i][1], font = ('Calibri', 15), fg = 'black',
497                      bg = 'light green').place(x = 400, y = y1)
498     y1 += 40
499 def doneButtonClick():
500     lb.destroy()
501     doneButton = Button(lb, text='DONE', font= ('Calibri', 20, 'bold'), bg = '#1d5e2f',
502                      fg = 'white', command=doneButtonClick).place(x=225, y=y1+20,
503                      height = 50, width= 100)
504
505 lbButton = Button(window4, text='LEADER BOARD', font= ('Calibri', 20, 'bold'),
506                 bg = '#ce402a', fg = 'white', command=leaderboard).place(x=100,
507                 y=30, height = 50, width= 200)
508
509 def endButtonClick():
510     window4.destroy()
511     endButton = Button(window4, text='END', font= ('Calibri', 20, 'bold'), bg = '#ce402a',
512                      fg = 'white', command=endButtonClick).place(x=100, y=110,
513                      height = 50, width= 200)
514
515 finishButton = Button(window3, text = 'FINISH', font= ('Calibri', 20, 'bold'),
516                     bg = '#e0d8c3', fg = 'black', command=open_window4).place(x = 640,
517                     y = 400, height = 40)
518
519 nextButton = Button(window3, text = 'NEXT', font= ('Calibri', 20, 'bold'), bg = '#e0d8c3',
520                  fg = 'black', command=nextButtonClick).place(x = 640, y = 320, height = 40)
521 window3.mainloop()
522 playButton = Button(window2, text='PLAY', font= ('Calibri', 30, 'bold'), bg = '#00537f',
523                  fg = 'white', command=constraint).place(x = 320, y = 375, height = 60)
524 window2.mainloop()
525 startButton = Button(window1, text='START', font= ('Calibri', 30, 'bold'), bg = '#f7be00',
526                  fg = 'white', command = open_window2).place(x = 325, y = 300, height = 60)
527 window1.mainloop()

```

SYSTEM DESIGN AND DEVELOPMENT

INPUT-OUTPUT SCREEN



player input

ENTER PLAYER NAMES

player 1:

player 2:

PLAY

player input

Error
Warning: names cannot be empty
[Try Again](#)

ENTER NAMES

player 1:

player 2:

PLAY

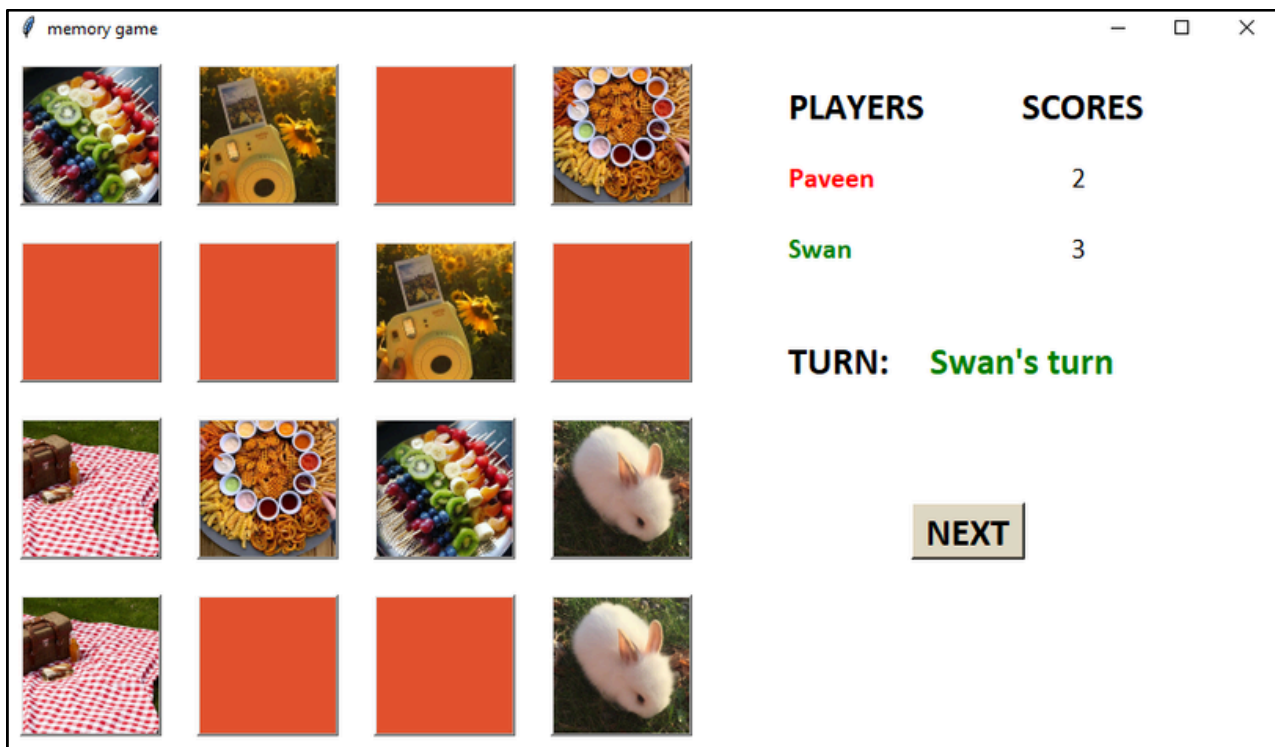
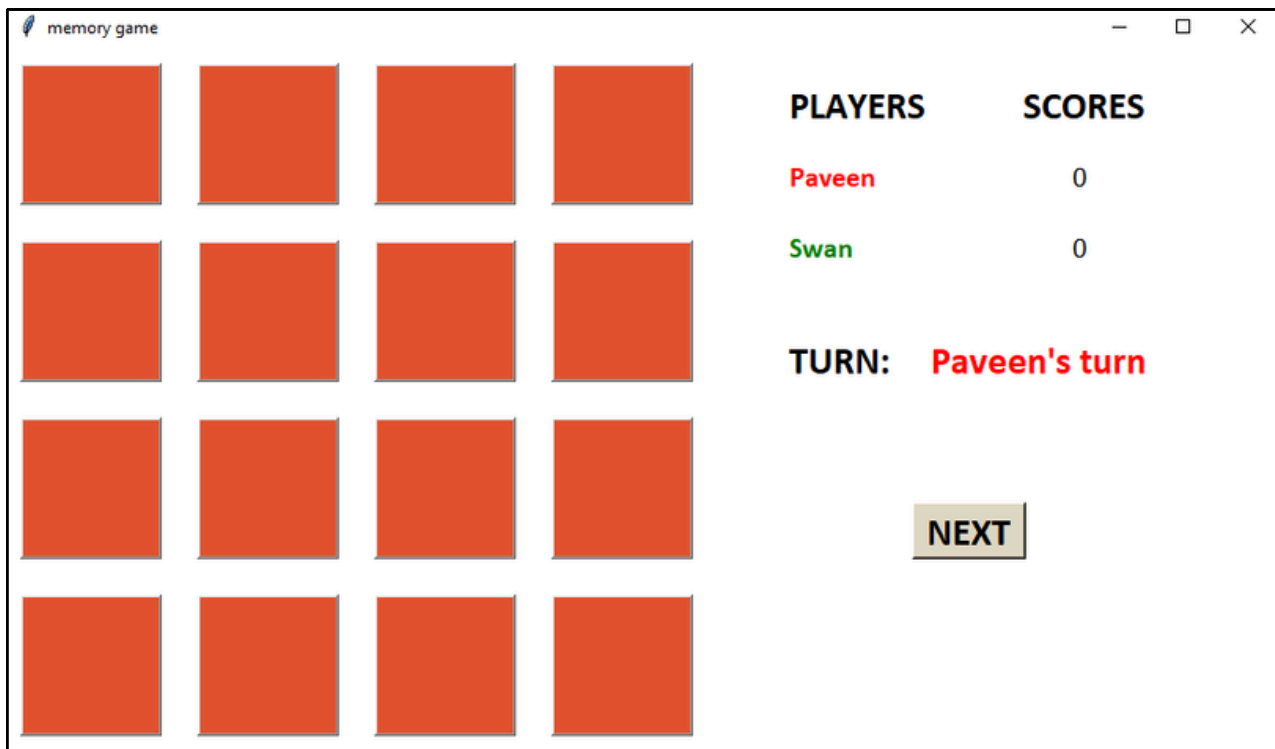
player input

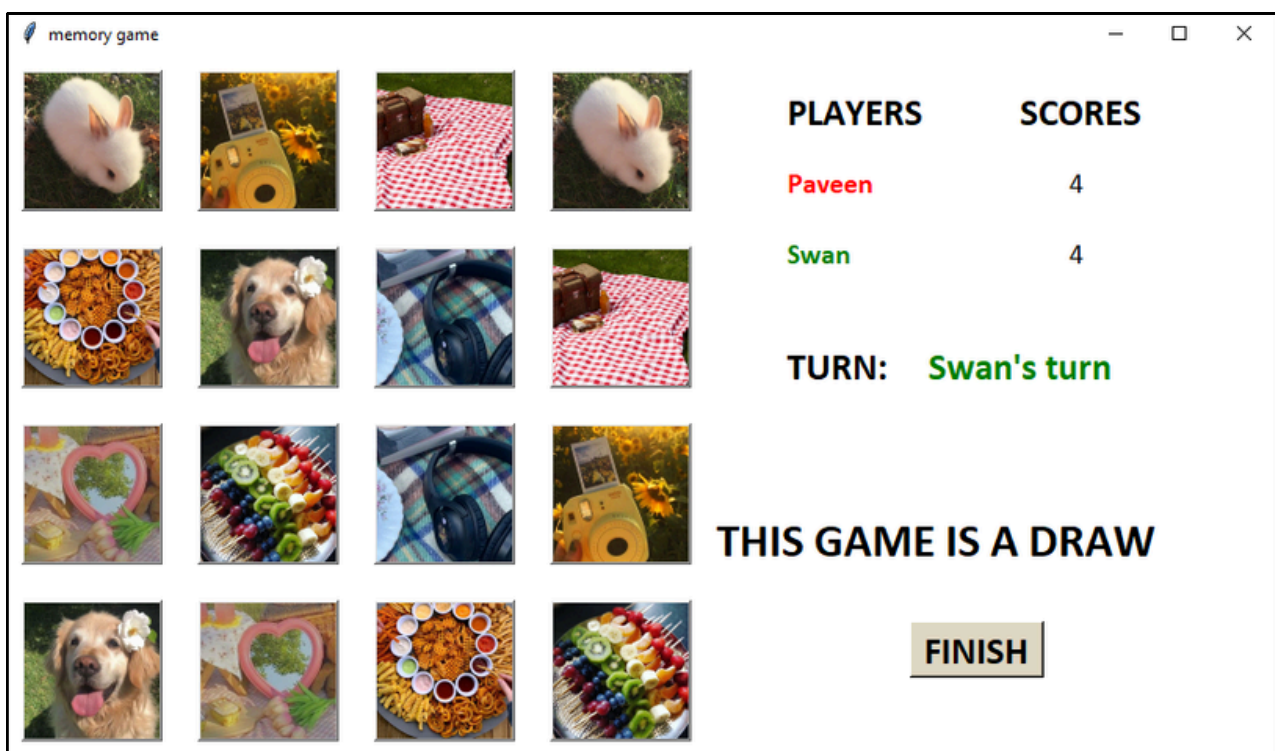
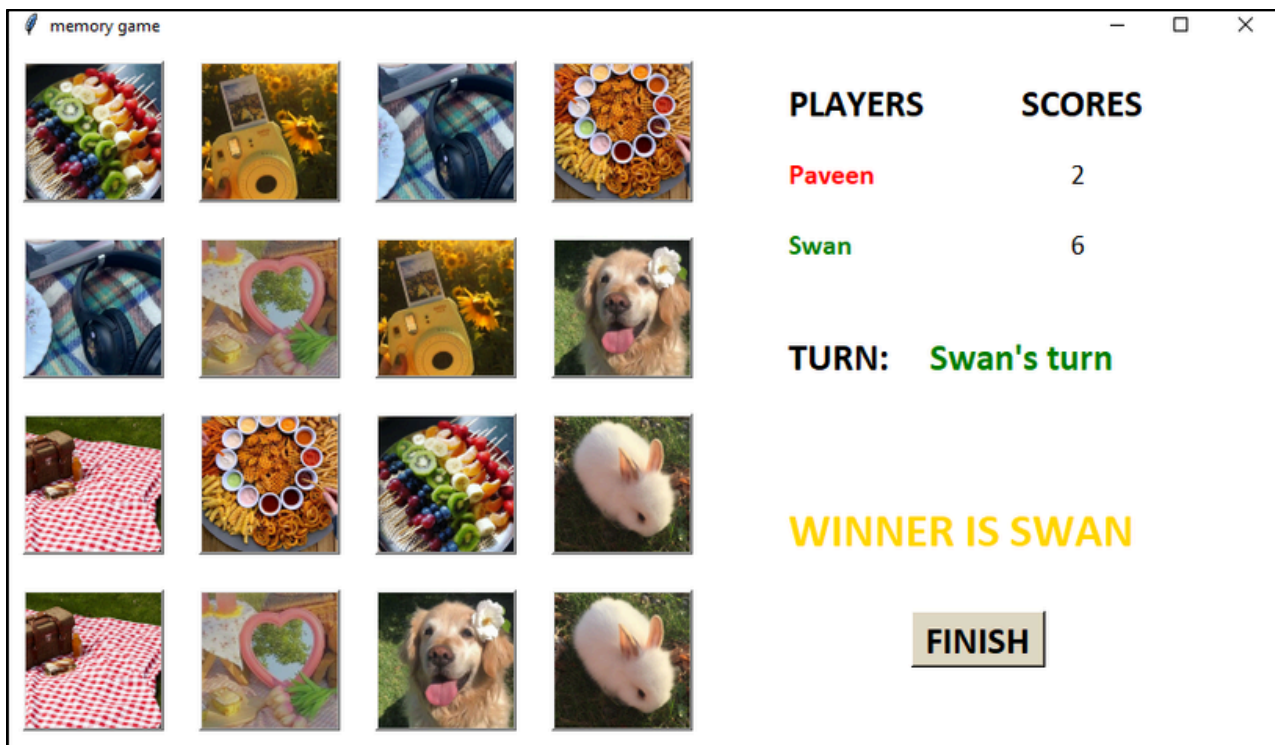
ENTER PLAYER NAMES

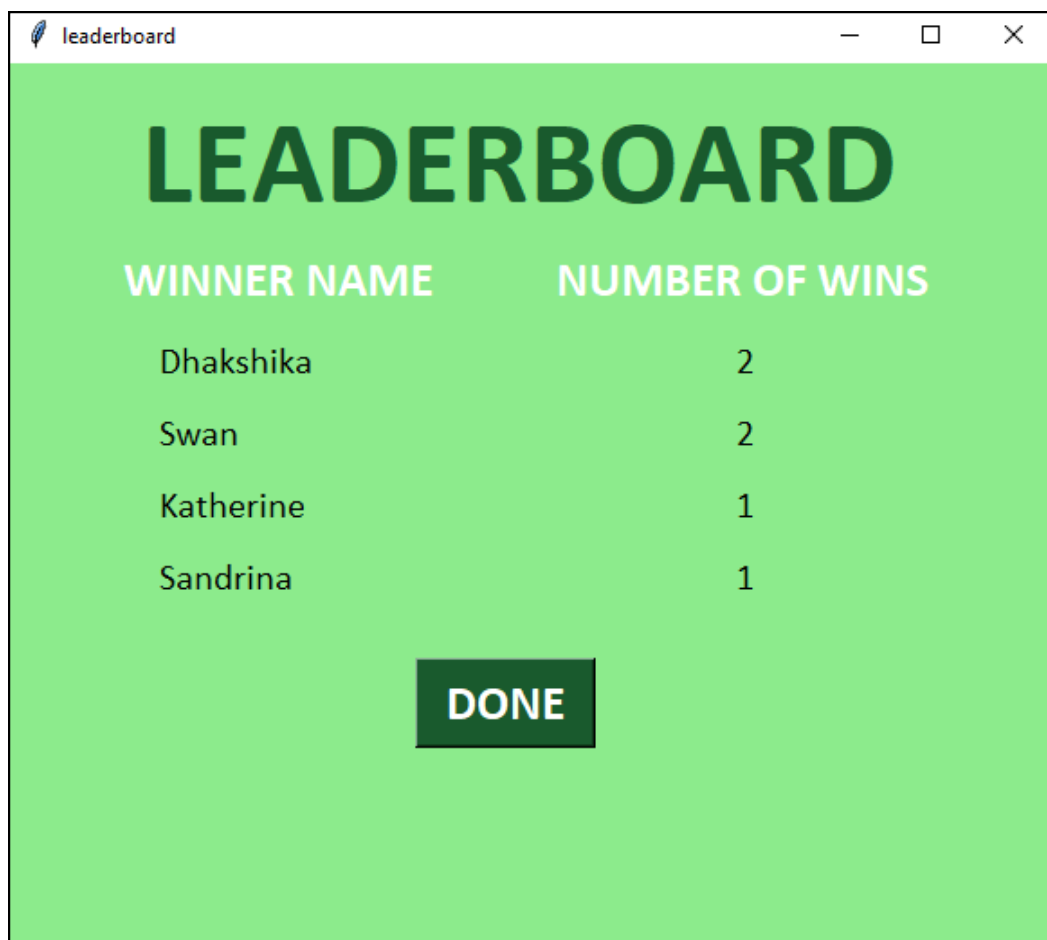
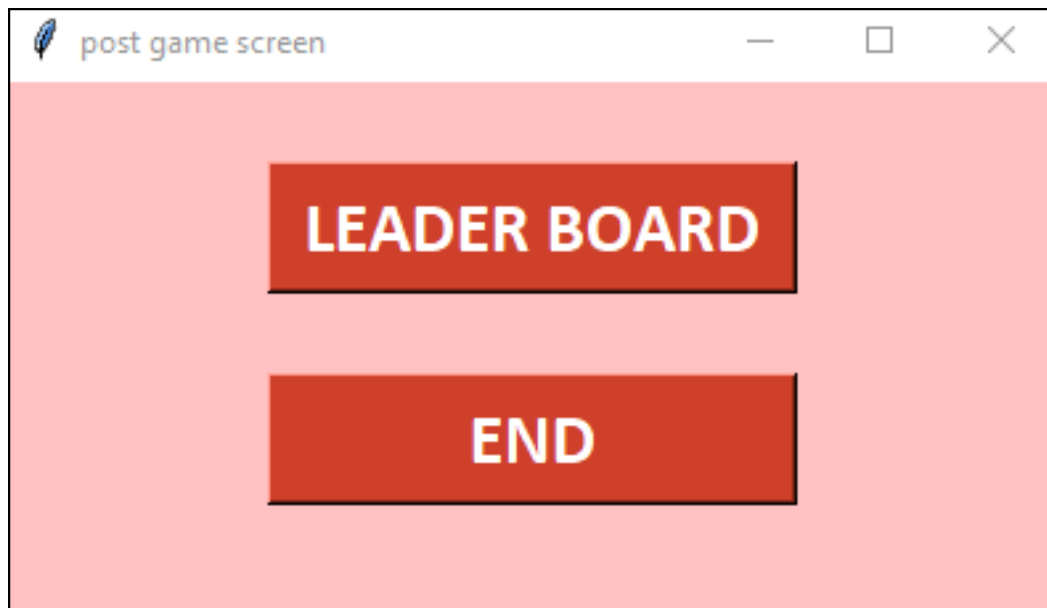
player 1:

player 2:

PLAY







CONCLUSION

In conclusion, our "Concentration" card game project offers an engaging and educational experience that enhances cognitive abilities such as memory, concentration, and problem-solving skills. By combining entertainment with learning, the game provides a fun platform for players of all ages to strengthen their mental faculties. The GUI interface ensures ease of use, making the game accessible to a broad audience. Tracking player performance through a live score counter and leaderboard adds a competitive edge, encouraging players to improve their skills. Additionally, the potential for future enhancements—such as customizable card pairs, multiplayer support, and various game modes—ensures that the game will remain challenging and enjoyable. This project is not only beneficial for students but also for educators, parents, and seniors seeking to maintain mental agility. Ultimately, the game is designed to foster cognitive development in a non-stressful, interactive environment. With its educational value and potential for customization, "Concentration" can be a powerful tool for learning and entertainment.

BIBLIOGRAPHY

<https://www.geeksforgeeks.org/>

<https://www.youtube.com/@Codemycom>